



UANL

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN



FIME

FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA

UNIVERSIDAD AUTÓNOMA DE NUEVO LEÓN
FACULTAD DE INGENIERÍA MECÁNICA Y ELÉCTRICA
LENGUAJES DE PROGRAMACIÓN Y LAB
DOCENTE: ING. SELENE GUADALUPE PINAL TORRES
ACTIVIDAD FUNDAMENTAL 5
EQUIPO: 04

Matricula	Nombre	Carrera
2103677	Ruedas Vazquez Jordan Emanuel	ITS
2042781	Enríquez Nungaray Héctor Miguel	IAS
1975208	Hernández Silva Carlos Yahir	ITS
2054413	Heredia López Betsy Aracely	IAS

MONTERREY, NUEVO LEÓN

INTRODUCCIÓN

Los lenguajes de programación son herramientas fundamentales que permiten a los desarrolladores comunicarse con las computadoras y escribir instrucciones que la máquina pueda ejecutar. Estos lenguajes permiten expresar algoritmos y soluciones a problemas de forma estructurada y comprensible. Para que una computadora entienda estas instrucciones, el código escrito en un lenguaje de alto nivel (entendido por los programadores) debe ser traducido a un lenguaje de bajo nivel (comprensible por la máquina). Este proceso de traducción es llevado a cabo por dos tipos de herramientas clave: los compiladores y los intérpretes.

Un compilador toma el código fuente completo y lo convierte en un archivo ejecutable en un solo paso, lo que permite que el programa se ejecute de forma más rápida y eficiente. Por otro lado, un intérprete traduce y ejecuta el código fuente línea por línea, lo que facilita la prueba y depuración de programas, pero con un costo en términos de velocidad de ejecución.

Ambos, compiladores e intérpretes, juegan un papel crucial en el ciclo de vida de desarrollo de software y en la ejecución de programas, y la elección entre uno y otro depende de diversos factores como el tipo de aplicación, la necesidad de rendimiento y la facilidad de depuración.



DESARROLLO DE LOS PRIMEROS LENGUAJES



- Años 40-50: Lenguaje máquina y ensamblador.
- Años 50-60: FORTRAN (ciencia) y COBOL (negocios).
- Años 70: C, base de muchos lenguajes actuales.
- Años 80-90: Aparecen lenguajes orientados a objetos como C++ y lenguajes modernos como Python y Java.

PAPEL DE LOS LENGUAJES DE PROGRAMACIÓN

- Traducen ideas en instrucciones para la computadora.
- Permiten crear software, aplicaciones, sitios web, etc.
- Hacen más fácil el trabajo del programador.
- Sirven para resolver problemas específicos (científicos, comerciales, juegos, IA, etc.).

¿QUÉ CONSTITUYE UN BUEN LENGUAJE?

- -Fácil de aprender y entender.
- -Permite escribir código claro y mantenible.
- -Eficiente en uso de recursos.
- -Funciona en distintas plataformas (portabilidad).
- -Flexible: admite distintos estilos de programación.
- -Facilita detección y manejo de errores.

ATRIBUTOS DE UN BUEN LENGUAJE

Atributo	Descripción
Simplicidad	Sintaxis clara y directa.
Legibilidad	Fácil de entender.
Eficiencia	Código rápido y optimizado.
Portabilidad	Funciona en varios sistemas.
Modularidad	Permite dividir el programa en partes reutilizables.
Seguridad	Ayuda a prevenir errores graves.
Soporte	Buena documentación y comunidad activa.

SINTAXIS DE LENGUAJES DE PROGRAMACIÓN

La sintaxis en programación hace referencia a un conjunto de reglas, principios y convenciones que rigen la estructura y el orden de las palabras y símbolos en un lenguaje de programación específico. Estas reglas definen cómo se escriben las instrucciones y cómo se organizan las diferentes partes del código fuente.

La sintaxis juega un papel fundamental en la comprensión y ejecución correcta del código por parte del compilador o intérprete. Si un programador no sigue la sintaxis adecuada del lenguaje de programación que está utilizando, el código puede producir errores de sintaxis que impiden la compilación o ejecución del programa.

Cada lenguaje de programación tiene su propia sintaxis única, aunque muchos comparten elementos comunes y siguen una misma lógica de la programación. Por ejemplo, en el lenguaje de programación Python, la indentación se utiliza para delimitar bloques de código, mientras que en C o Java, se utilizan llaves {}.

Elementos de la sintaxis

Existen varios elementos importantes relativos a la sintaxis en un lenguaje de programación y son los siguientes:

Palabras reservadas: Son palabras que tienen un significado especial en el lenguaje de programación y se utilizan para realizar acciones específicas. Por ejemplo, "if", "else", "for", "while", "class", entre otras.

Identificadores: Son nombres que se utilizan para identificar variables, funciones, clases, módulos u otros objetos en el código. Los identificadores deben seguir ciertas reglas como no empezar con números o incluir espacios.

Operadores: Son símbolos utilizados para realizar operaciones matemáticas, de asignación o lógicas. Algunos ejemplos son "+", "-", "*", "/", "=", "&&", "||", entre otros. Destacan los operadores booleanos.

Delimitadores: Son caracteres utilizados para separar y agrupar diferentes elementos en el código. Algunos ejemplos son ";", "{", "}", "(", ")", "[", "]", entre otros.

Comentarios: Son textos utilizados para explicar o documentar el código. No afectan al funcionamiento del programa, pero ayudan a entender el código.

Palabras clave: Son palabras que tienen un significado específico en un lenguaje de programación y se utilizan para realizar acciones específicas. Por ejemplo, "import", "public", "private", "return", entre otras.

Diferencias de sintaxis en lenguajes de programación

CRITERIOS GENERALES DE LA SINTAXIS

La sintaxis puede variar significativamente entre diferentes lenguajes de programación, lo que significa que un programador debe conocer la sintaxis del lenguaje que está utilizando. Estas diferencias pueden incluir la forma en que se escriben las estructuras de control de flujo, la declaración de variables, la gestión de la memoria, entre otros aspectos. Por ejemplo:

Python: Es conocido por su sintaxis limpia y legible, que utiliza la indentación para delimitar bloques de código en lugar de utilizar llaves o palabras clave como `begin` y `end`.

C: Es un lenguaje de programación de bajo nivel que utiliza una sintaxis más compacta y cercana al hardware de la computadora. Requiere el uso explícito de punto y coma al final de cada declaración y utiliza llaves para delimitar bloques de código.

JavaScript: Es un lenguaje de programación ampliamente utilizado en el desarrollo web. Tiene una sintaxis similar a la de C, pero también incluye conceptos tomados de otros lenguajes como Java y Python. JavaScript es un lenguaje interpretado, lo que significa que se ejecuta en el navegador del usuario en lugar de en el servidor.

SQL (Structured Query Language): Es un lenguaje de consulta utilizado para interactuar con bases de datos relacionales. Tiene una sintaxis específica para realizar consultas, actualizaciones, inserciones y eliminaciones de datos en tablas de bases de datos.

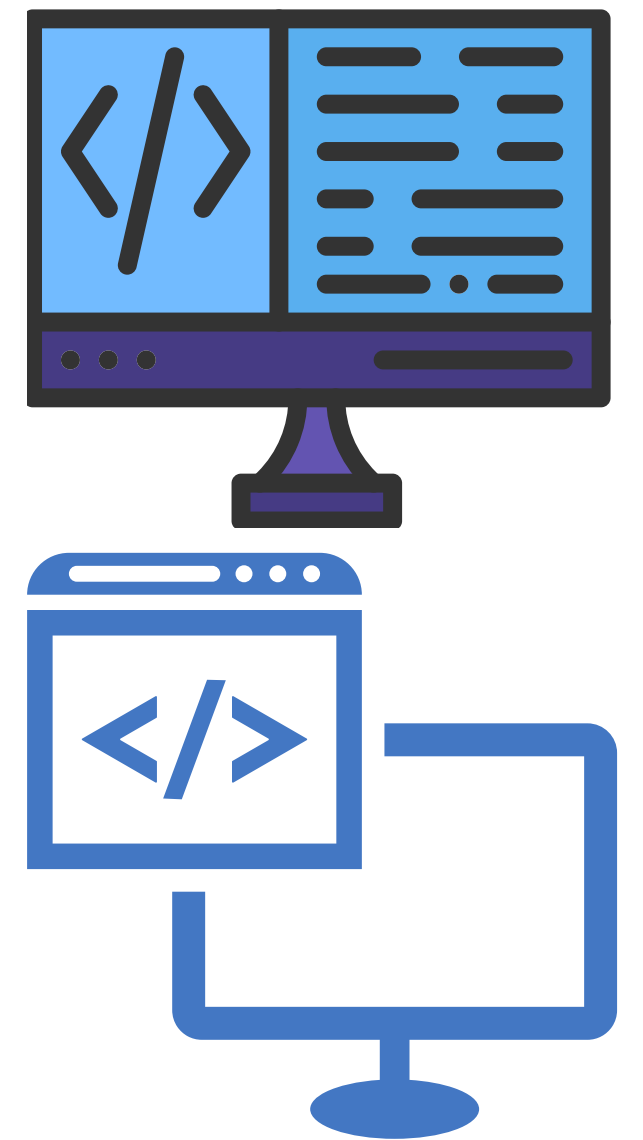
En resumen, la sintaxis es esencial para la escritura y comprensión del código en un lenguaje de programación. Los programadores deben familiarizarse con la sintaxis del lenguaje que están utilizando para poder escribir código correctamente y evitar errores de sintaxis.

ELEMENTOS SINTÁCTICOS DE UN LENGUAJE

Los elementos sintácticos de un lenguaje de programación son las unidades fundamentales que conforman su estructura y permiten escribir instrucciones comprensibles tanto para los programadores como para los compiladores e intérpretes. Uno de los elementos esenciales son las palabras clave, que son términos reservados del lenguaje utilizados para definir estructuras y ejecutar acciones específicas. Por ejemplo, en lenguajes como Java y Python, palabras clave como `if`, `else`, `for`, `while` y `return` permiten el control de flujo en un programa. Estas palabras clave no pueden ser utilizadas como nombres de variables o funciones, ya que tienen un significado especial dentro del lenguaje.

Otro elemento clave en la sintaxis de un lenguaje de programación son los identificadores, que representan nombres asignados a variables, funciones, clases y otros objetos dentro del código. Los identificadores deben seguir ciertas reglas, como no comenzar con números y no contener espacios en blanco. La elección de nombres descriptivos para los identificadores es una buena práctica de programación, ya que mejora la legibilidad del código y facilita su mantenimiento. Además, los operadores juegan un papel fundamental en la sintaxis de los lenguajes de programación, ya que permiten realizar operaciones matemáticas, lógicas y de asignación. Algunos ejemplos incluyen `+`, `-`, `*`, `/` para operaciones aritméticas y `&&`, `||`, `!` para operaciones lógicas.

Por último, los delimitadores y estructuras de control ayudan a organizar y estructurar el código de manera lógica. Los delimitadores, como `{}` en C y Java, se utilizan para definir bloques de código, mientras que en Python la indentación cumple esta función. Además, los lenguajes de programación incluyen comentarios, que permiten a los programadores documentar el código sin afectar su ejecución. Estos comentarios pueden ser de una sola línea (`//` en C y Java, `#` en Python) o de múltiples líneas (`/* ... */` en Java y C). La correcta utilización de estos elementos sintácticos es crucial para garantizar la claridad y eficiencia del código, permitiendo a los desarrolladores escribir programas estructurados y fáciles de entender.

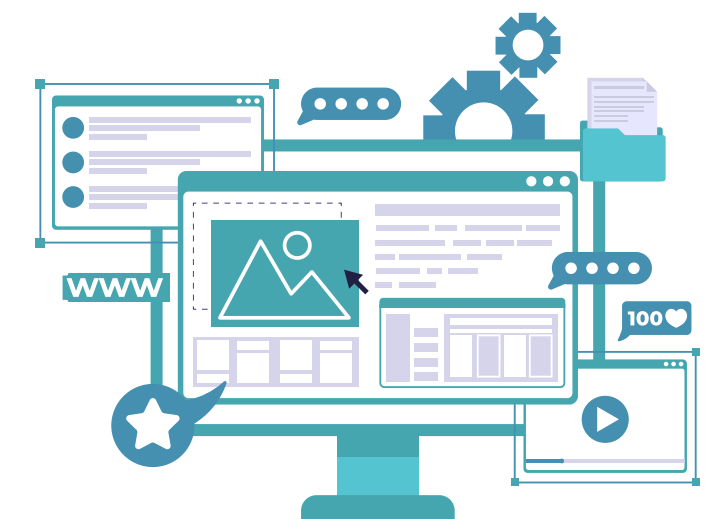


ESTRUCTURA DE CONJUNTO DE PROGRAMAS Y SUBPROGRAMAS

En la programación, un programa informático está compuesto por una serie de instrucciones organizadas en una estructura lógica que permite ejecutar tareas específicas. Para facilitar su desarrollo y mantenimiento, los programas suelen dividirse en módulos o subprogramas, los cuales permiten reutilizar código y mejorar la legibilidad del software. La estructura de un programa varía según el lenguaje utilizado, pero generalmente sigue un esquema jerárquico en el que existen bloques principales y secundarios. Esta organización es fundamental para garantizar la modularidad y escalabilidad de las aplicaciones, permitiendo que distintos equipos de desarrollo trabajen simultáneamente en diferentes partes del programa sin afectar su funcionalidad global.

Los subprogramas son fragmentos de código que cumplen una función específica dentro de un programa mayor. Pueden clasificarse en funciones y procedimientos, dependiendo de si retornan un valor o no. Las funciones son bloques de código que realizan una operación y devuelven un resultado, mientras que los procedimientos ejecutan instrucciones sin devolver un valor concreto. En lenguajes como C, Java y Python, el uso de funciones es fundamental para estructurar el código de manera eficiente y evitar la repetición de instrucciones. Además, los subprogramas pueden recibir parámetros de entrada y trabajar con datos específicos sin necesidad de depender de variables globales, lo que mejora la organización y reutilización del código.

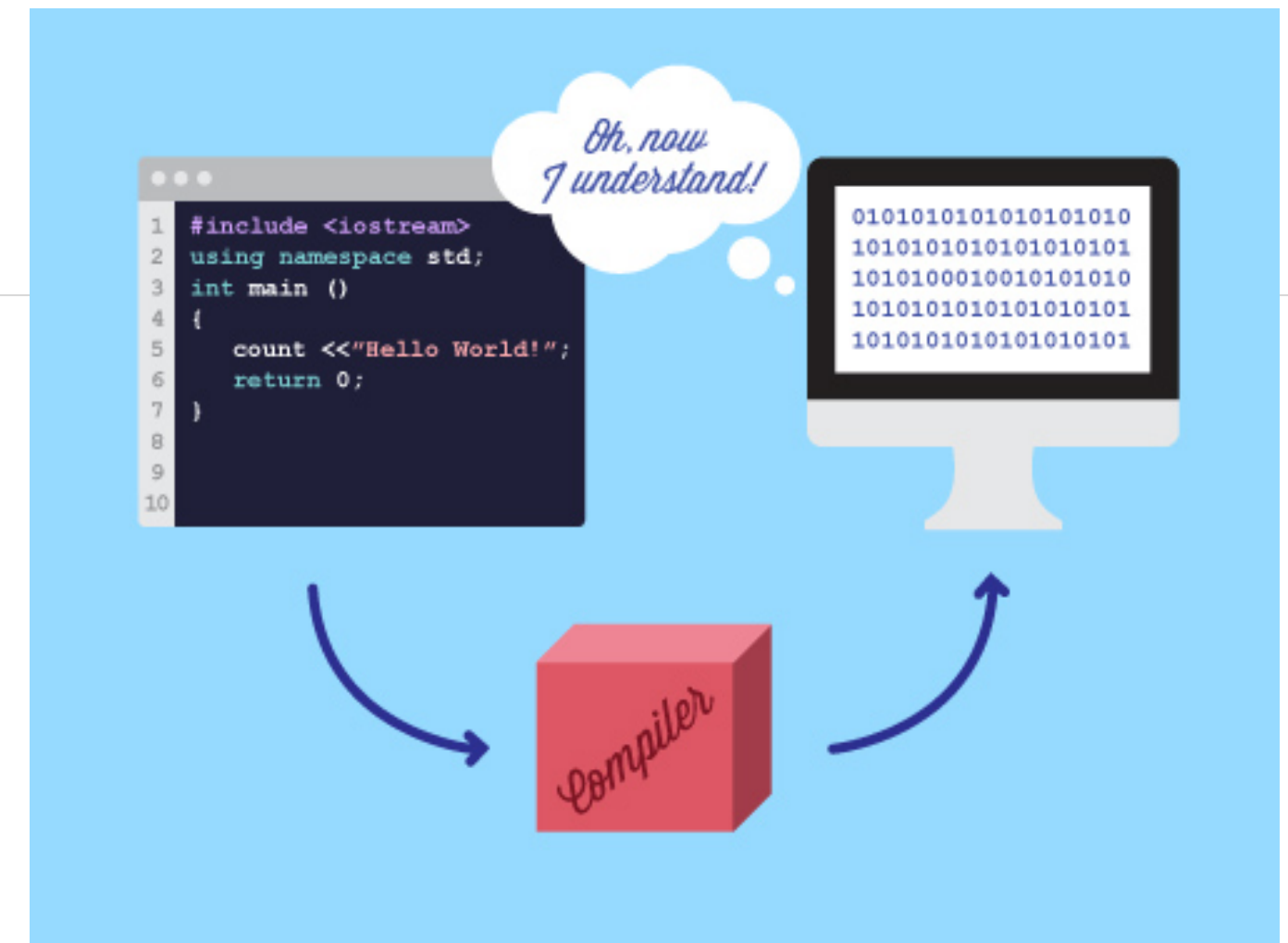
Un aspecto clave en la estructura de los programas es la correcta implementación del control de flujo, que determina el orden en que se ejecutan las instrucciones. Este control se logra mediante estructuras de decisión (como if-else), bucles (for, while), y llamadas a subprogramas. La adecuada utilización de estas estructuras permite desarrollar programas más eficientes, fáciles de depurar y escalables. En aplicaciones grandes, la división del código en múltiples subprogramas organizados en módulos independientes facilita su mantenimiento y actualización, permitiendo que cada módulo pueda ser probado y mejorado sin afectar el funcionamiento del resto del sistema.



COMPILADORES E INTÉRPRETES

Los compiladores e intérpretes son herramientas fundamentales en la ejecución de programas, ya que permiten transformar el código fuente escrito por los programadores en instrucciones comprensibles para la computadora.

Un **compilador** es un software que traduce el código fuente completo de un programa a código máquina antes de su ejecución, generando un archivo ejecutable independiente del código original. Ejemplos de compiladores incluyen GCC (para C y C++), el compilador de Java (javac) y el de Pascal (Free Pascal). Este método tiene la ventaja de optimizar la ejecución del programa, ya que la traducción ocurre solo una vez, permitiendo que el software se ejecute de manera rápida y eficiente. Sin embargo, la detección de errores en un programa compilado solo ocurre después de haber procesado todo el código, lo que puede dificultar la depuración en algunas circunstancias.



COMPILADORES E INTÉRPRETES

Por otro lado, un **intérprete** traduce y ejecuta el código fuente línea por línea en tiempo de ejecución, sin generar un archivo ejecutable independiente. Esto permite una depuración más sencilla, ya que los errores se identifican y corrigen inmediatamente sin necesidad de recompilar todo el código. Sin embargo, los programas interpretados suelen ser más lentos en ejecución, ya que cada instrucción debe traducirse y ejecutarse en tiempo real. Ejemplos de lenguajes interpretados incluyen Python, JavaScript y PHP, los cuales se utilizan ampliamente en aplicaciones web y entornos de scripting debido a su flexibilidad y facilidad de uso.

Algunas tecnologías modernas combinan ambos enfoques mediante **compilación en tiempo de ejecución (Just-In-Time, JIT)**, donde el código fuente se traduce parcialmente antes de la ejecución y se optimiza mientras el programa está en uso. Java y C# son ejemplos de lenguajes que utilizan este método, lo que les permite ofrecer un buen equilibrio entre rendimiento y flexibilidad. La elección entre un compilador o un intérprete depende de factores como la necesidad de rendimiento, la facilidad de depuración y la naturaleza del software desarrollado. En general, los compiladores son preferidos para aplicaciones que requieren alta velocidad de ejecución, mientras que los intérpretes son ideales para entornos de desarrollo dinámicos y pruebas rápidas.



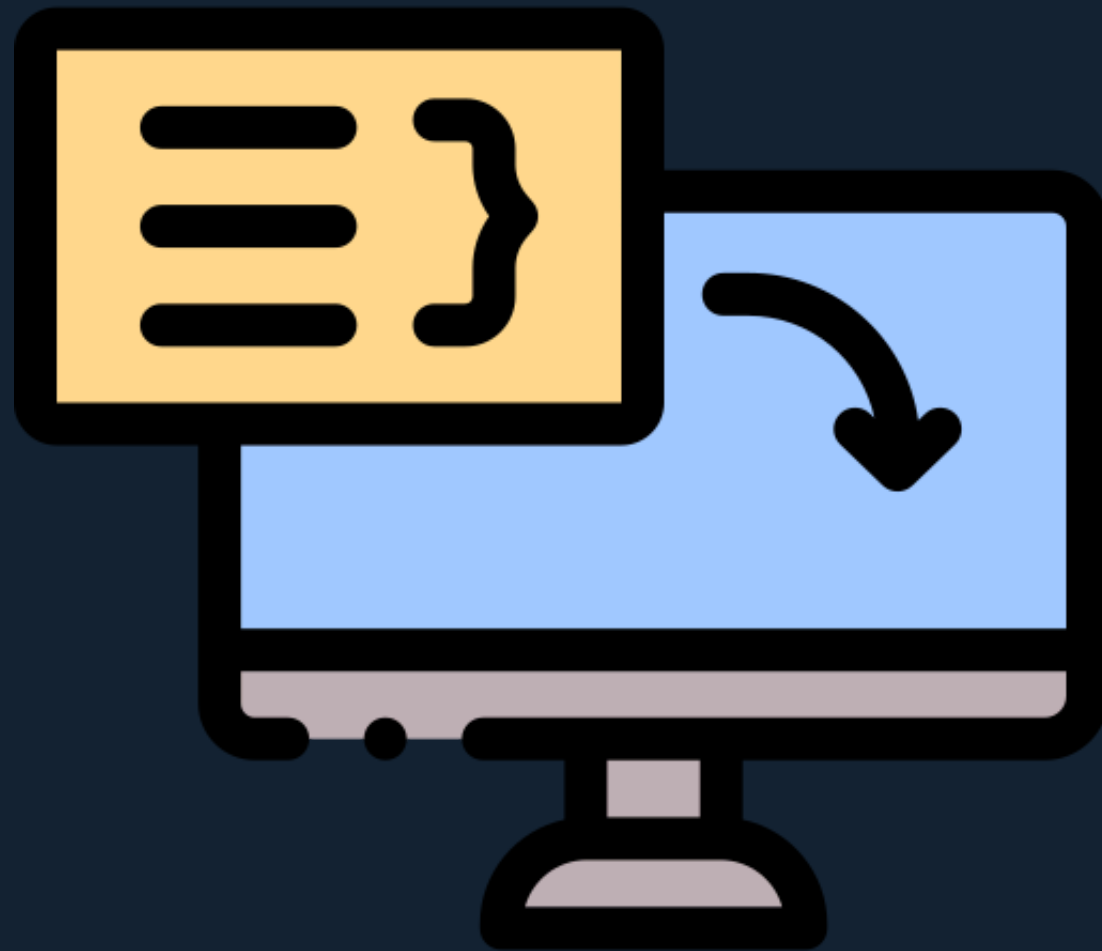
Compilador Java



```
1:  istore_1
2:  iload_1
3:  sipush 1000
6:  if_icmpge 44
9:  iconst_2
10: istore_2
11: iload_2
12: iload_1
13: if_icmpge 31
16: iload_1
17: iload_2
18: irem      # remainder
19: ifne 25
22: goto 38
```

Intérprete JVM





Características	Interprete	Compilador
Descripción	Programa que procesa el código fuente línea por línea durante la ejecución del software. Actúa como una interfaz entre el código y el procesador, traduciendo cada instrucción en tiempo real.	Programa que traduce todo el código fuente a código máquina antes de ejecutarlo. En muchos casos, primero convierte el código en un código intermedio antes de generar la versión final en código máquina.
Ventajas	-Permite ejecutar el programa de inmediato sin esperar a la compilación completa. - Facilita la depuración porque los errores se detectan línea por línea. - Desarrollo más sencillo y flexible. - Bajo coste de desarrollo. - Ideal para pruebas y modificaciones rápidas.	-Genera código ejecutable optimizado y rápido. - No requiere el código fuente para ejecutar el programa. - Mayor eficiencia en la ejecución. - Puede optimizar el orden de ejecución de las instrucciones. - Utiliza menos recursos en tiempo de ejecución.
Desventajas	-Velocidad de ejecución más lenta porque traduce en tiempo real. - Requiere que el código fuente esté disponible en todo momento. - Consume más recursos durante la ejecución. - Menor eficiencia de traducción. - No es ideal para aplicaciones que requieren alto rendimiento	-Tiempo de compilación alto. - Si hay errores, se deben corregir antes de ejecutar el programa. - Menos flexible para pruebas y modificaciones rápidas. - Coste de desarrollo más alto. - Cualquier cambio en el código requiere recompilación.
Ejemplos	-Python (CPython) - PHP - Ruby - Perl - BASIC	-C - C++ - Pascal - Kotlin Compiler - Turbo Pascal

CONCLUSIÓN GENERAL



Los lenguajes de programación han evolucionado a lo largo de la historia para facilitar la interacción entre los programadores y las computadoras, permitiendo el desarrollo de software más eficiente y accesible. Desde los primeros lenguajes de bajo nivel como el ensamblador hasta las modernas tecnologías de alto nivel como Python y JavaScript, el objetivo ha sido siempre mejorar la productividad y la capacidad de resolución de problemas. Gracias a los avances en los paradigmas de programación, hoy en día existen lenguajes especializados para diversas áreas, desde la inteligencia artificial hasta el desarrollo web, cada uno optimizado para tareas específicas. Uno de los aspectos más importantes en la programación es la manera en que los lenguajes traducen las instrucciones humanas a código máquina. En este sentido, los compiladores e intérpretes juegan un papel crucial en la ejecución del software. Mientras que los compiladores generan programas altamente optimizados y rápidos, los intérpretes brindan mayor flexibilidad y facilidad de depuración. La elección entre uno y otro depende de los requisitos del proyecto y del tipo de aplicación que se desee desarrollar. Además, la evolución de técnicas como la compilación JIT ha permitido combinar las ventajas de ambos métodos, optimizando el rendimiento sin sacrificar la flexibilidad. En conclusión, el estudio de los lenguajes de programación, sus estructuras y las herramientas que permiten su ejecución es fundamental para cualquier desarrollador. Comprender la sintaxis, los atributos de un buen lenguaje y el papel de los compiladores e intérpretes permite tomar decisiones informadas sobre qué herramientas utilizar en cada proyecto. A medida que la tecnología sigue avanzando, la programación continuará evolucionando para hacer que el desarrollo de software sea más eficiente, accesible y adaptado a las necesidades cambiantes del mundo digital.

CONCLUSIONES INDIVIDUALES

ENRÍQUEZ NUNGARAY HÉCTOR MIGUEL

Los lenguajes de programación son la base del desarrollo tecnológico. Han evolucionado para adaptarse a nuevas necesidades, facilitando la creación de software eficiente y seguro. Su futuro seguirá marcado por la innovación y la adaptación a nuevas tecnologías.

HEREDIA LÓPEZ BETSY ARACELY

En mi opinión, los lenguajes de programación y los intérpretes son componentes esenciales en el desarrollo de software, cada uno con su propio propósito y beneficios. Los lenguajes de programación ofrecen una forma estructurada para que los programadores se comuniquen con las computadoras, permitiendo la creación de aplicaciones y sistemas que van desde lo simple hasta lo más complejo. La evolución de estos lenguajes ha permitido que sean cada vez más accesibles y poderosos, adaptándose a nuevas necesidades y tecnologías. Los intérpretes, por su parte, son herramientas clave que permiten ejecutar programas de manera flexible y eficiente, especialmente en lenguajes dinámicos o en entornos de desarrollo interactivo. Aunque pueden ser más lentos que los compiladores, su capacidad para ejecutar código de manera inmediata facilita el proceso de prueba y depuración, lo que resulta valioso en muchas situaciones de desarrollo.

HERNÁNDEZ SILVA CARLOS YAHIR

Como vimos los lenguajes de programación han sido de mucha ayuda desde los años de su creación tanto para el desarrollo de nuevas tecnologías y creación de más lenguajes. Nos ayudan para una evolución de la comunicación de los humanos y las computadoras. Existen muchos lenguajes de programación, pero no muchos cumplen con lo esencial su eficiencia, que tenga un fácil uso y pero también varia dependiendo que quieres hacer en tu lenguaje.

2103677 RUEDAS VÁZQUEZ JORDAN EMANUEL

El estudio de los lenguajes de programación, su evolución y las herramientas que permiten su ejecución es esencial para comprender el impacto que tienen en el desarrollo de software y la informática en general. A lo largo de este análisis, se ha podido observar cómo los lenguajes han pasado de ser sistemas complejos y de bajo nivel a herramientas intuitivas y versátiles, diseñadas para adaptarse a las necesidades actuales de la tecnología. Además, la diferencia entre compiladores e intérpretes resalta la importancia de elegir la metodología adecuada según el propósito del software, priorizando ya sea el rendimiento o la flexibilidad.

REFERENCIAS BIBLIOGRAFICAS

CAIRO, O. (2024). ALGORITMOS Y ESTRUCTURAS DE DATOS. EDITORIAL UNIVERSITARIA.

GAUCHAT, J. D. (2017). EL GRAN LIBRO DE HTML5, CSS3 Y JAVASCRIPT. MARCOMBO.

MOZILLA DEVELOPER NETWORK. (2024). JAVASCRIPT GUIDE.

[HTTPS://DEVELOPER.MOZILLA.ORG/ES/DOCS/WEB/JAVASCRIPT/GUIDE](https://developer.mozilla.org/es/docs/web/javascript/guide)

TYPESCRIPT HANDBOOK. (2024). BASIC TYPES.

[HTTPS://WWW.TYPESCRIPTLANG.ORG/DOCS/HANDBOOK/BASIC-TYPES.HTML](https://www.typescriptlang.org/docs/handbook/basic-types.html)

W3C. (2024). HTML SPECIFICATIONS. [HTTPS://WWW.W3.ORG/TR/HTML/](https://www.w3.org/tr/html/)

PRESSMAN, R. S. (2014). INGENIERÍA DEL SOFTWARE: UN ENFOQUE PRÁCTICO (8ª ED.). MCGRAW-HILL.

SOMMERVILLE, I. (2015). SOFTWARE ENGINEERING (10TH ED.). PEARSON.

CORMEN, T. H., LEISERSON, C. E., RIVEST, R. L., & STEIN, C. (2009). INTRODUCTION TO ALGORITHMS (3RD ED.).

FLANAGAN, D. (2011). JAVASCRIPT: THE DEFINITIVE GUIDE. O'REILLY MEDIA.

KOENIGSBERGER, G. (1998). INTERNET: ARQUITECTURA Y PROTOCOLO TCP/IP. PEARSON EDUCACIÓN.

SUBRAMANIAN, V. (2018). PROGRAMMING JAVASCRIPT APPLICATIONS: ROBUST WEB ARCHITECTURE WITH NODE, HTML5, AND MODERN JS LIBRARIES. O'REILLY MEDIA.

TANENBAUM, A. S. (2019). STRUCTURED COMPUTER ORGANIZATION (6TH ED.). PEARSON.

SEBESTA, R. W. (2020). CONCEPTS OF PROGRAMMING LANGUAGES (12TH ED.). PEARSON.

ULLMAN, J. D., & AHO, A. V. (2007). COMPILERS: PRINCIPLES, TECHNIQUES, AND TOOLS (2ND ED.). ADDISON-WESLEY.

MCCONNELL, S. (2004). CODE COMPLETE: A PRACTICAL HANDBOOK OF SOFTWARE CONSTRUCTION (2ND ED.). MICROSOFT PRESS.

THANK
YOU